Faster sorting algorithms discovered using deep reinforcement learning

Mankowitz, D.J., Michi, A., Zhernov, A. et al.
Google DeepMind

Nature, 2023

Introduction

The Problem:
A lot of work has gone into developing faster sorting algorithms, but we've hit a wall. Can a machine find new insights that humans have missed?

The Approach:
AlphaZero (2017) has been used to discover new insights in games: Chess, Shogi, Go.
What if we treat sorting as a game and use AlphaZero to discover new algorithms?
Enter **AlphaDev**

Contributions:

- Frame sorting as a single-player game
- Use DRL to solve the game
- Translate solutions back into code and isolate the insights

Outline

- Problem Formulation
- DRL Algorithm (AlphaZero inspired)
- Results
- Discoveries

Goal

Goal: Build an assembly program to solve a specific problem that is both:

- **Correct:** Produces the correct output for all input (for all the tried inputs).
- **Fast:** Minimize instruction length or execution latency (as measured on real hardware)

Chosen Problems:

- **Fixed Sorts:** Algorithm to sort a fixed number of elements
- **Variable Sorts:** Algorithm to sort up to a fixed number of elements (e.g., 3, 4, or 5)

Input:

- A correct algorithm
- Environment to execute the algorithm and provide feedback on correctness and latency

a

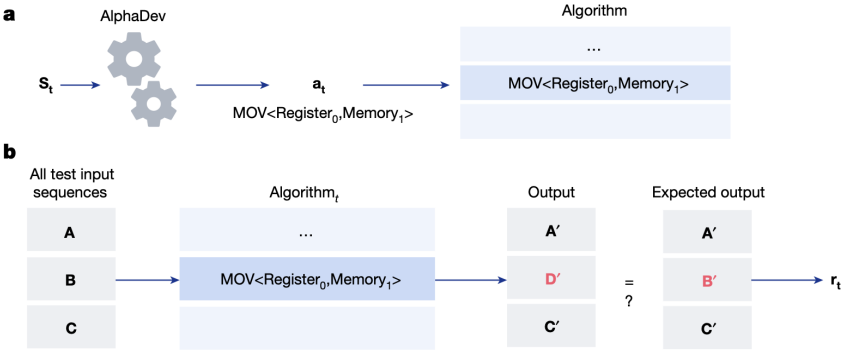
```c
void variable_sort_2(int length, int *a)
{
  // Determine the number of elements
  switch (length)
  {
    case 0:
    case 1:
      // Exit if less than 2 elements
      return;
    case 2:
      // Below routine sorts 2 elements
      int tmp = a[0];
      a[0] = (a[1] < a[0]) ? a[1] : a[0];
      a[1] = (a[1] < tmp) ? tmp : a[1];
      return;
  }
}
```

b

```asm
variable_sort_2(int, int*):
        ; Determine the number of elements
        cmp     $2, %edi
        ; Exit if less than 2 elements
        jne     .Label
        ; Below routine sorts 2 elements
        mov     (%rsi), %eax
        mov     4(%rsi), %ecx
        cmp     %eax, %ecx
        mov     %eax, %edx
        cmovl   %ecx, %edx
        mov     %edx, (%rsi)
        cmovg %ecx, %eax
        mov     %eax, 4(%rsi)

.Label:
        retq
```

AssemblyGame

- Single-player Reinforcement Learning environment: **AssemblyGame**.
- **State:** The current partial program and the state of memory/registers. $S_t =< P_t, Z_t >$
- **Action:** Append a valid x86 assembly instruction. A_t
- **Reward:** Correctness reward and latency reward based on the partial program. R_t



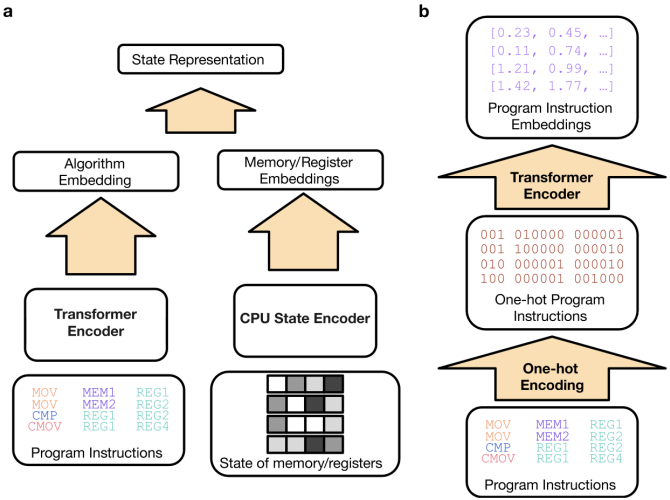
Restricted Space

To keep the search space small, they limit the problem considerably:

- Only a few memory locations and registers are given access (addressable discretely).
- Only a tiny subset of x86 instructions are allowed
- Action pruning rules: e.g., memory locations read in incremental order, memory read and written to only once, no consecutive compares etc.

Instruction	Description
mov<A,B>	Move a value from source (A) to destination (B).
cmovX<A,B>	Conditionally move the value from A to B if a register flag is set where X can be L (Less than), G (Greater than), LE (Less than or equal) or GE (Greater than or equal).
cmp<A,B>	Compare whether the values A and B are equal and set a register flag accordingly.
jX<A,B>	Jump to a different part of the program defined by the label A. Here X can correspond to values such as NE (Not Equal), E (Equal) or can be left blank, in which case the jump will be executed regardless of the register flag. This instruction is used when there is branching such as if-else or switch-case statements in the C++ function.

Architecture



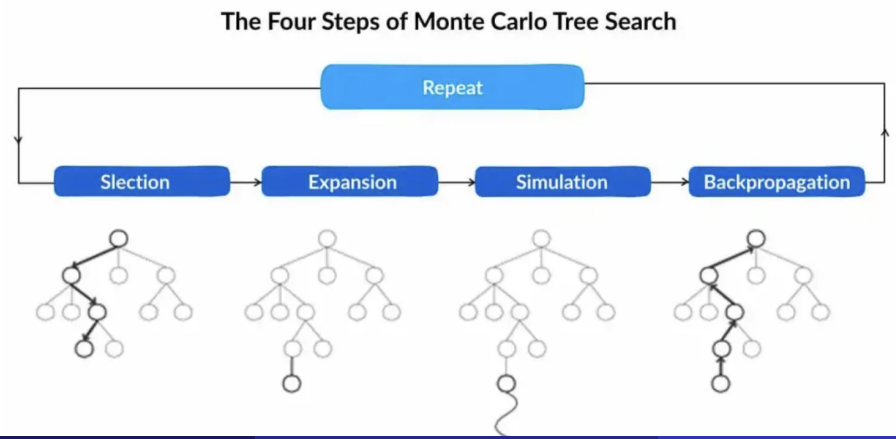
Algorithm

- **Encoder:** A Transformer-based architecture that encodes the current state of the program and memory into a latent representation.
- **Value and Policy Heads:** The value head predicts the expected reward (correctness and latency) from the current state, while the policy head outputs a probability distribution over the next valid assembly instructions.
- **Dynamics Model (Optional):** Predicts the next state and reward given the current state and action, allowing for lookahead without executing on hardware.

The training loop follows Monte Carlo Tree Search (MCTS) to explore the search space of possible programs, guided by the policy and value predictions from the neural network. The reward signal is based on the correctness and latency of the generated program, as evaluated on real hardware.

Monte Carlo Tree Search (MCTS)

- Building an Asymmetric Search Tree focused on promising branches
- Maintains statistics at each node: visit count, total value, prior probability
- Run simulations to explore the search space



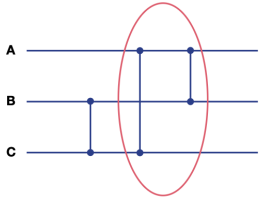
Results

(a) Algorithm	AlphaDev	Human benchmarks
	Length	**Length**
Sort 3	17	18
Sort 4	28	28
Sort 5	42	46
VarSort3	21	33
VarSort4	37	66
VarSort5	63	115
VarInt	27	31

(b) Algorithm	AlphaDev	Human benchmarks
	Latency ± (lower, upper)	**Latency ± (lower, upper)**
VarSort3	236,498 ± (235,898, 236,887)	246,040 ± (245,331, 246,470)
VarSort4	279,339 ± (278,791, 279,851)	294,963 ± (294,514, 295,618)
VarSort5	312,079 ± (311,515, 312,787)	331,198 ± (330,717, 331,850)
VarInt	97,184 ± (96,885, 97,847)	295,358 ± (293,923, 296,297)
Competitive	75,973 ± (75,420, 76,638)	86,056 ± (85,630, 86,913)

Novelty: Swap Move

a



b Original

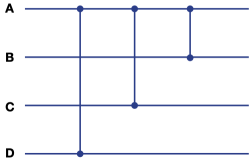
```
Memory[0] = A
Memory[1] = B
Memory[2] = C

mov Memory[0] P  // P = A
mov Memory[1] Q  // Q = B
mov Memory[2] R  // R = C

mov R S
cmp P R
cmovg P R  // R = max(A, C)
cmovl P S  // S = min(A, C)
mov S P    // P = min(A, C)
cmp S Q
cmovg Q P  // P = min(A, B, C)
cmovg S Q  // Q = max(min(A, C), B)

mov P Memory[0]  // = min(A, B, C)
mov Q Memory[1]  // = max(min(A, C), B)
mov R Memory[2]  // = max(A, C)
```

c AlphaDev

```
Memory[0] = A
Memory[1] = B
Memory[2] = C

mov Memory[0] P  // P = A
mov Memory[1] Q  // Q = B
mov Memory[2] R  // R = C

mov R S
cmp P R
cmovg P R  // R = max(A, C)
cmovl P S  // S = min(A, C)

cmp S Q
cmovg Q P  // P = min(A, B)
cmovg S Q  // Q = max(min(A, C), B)

mov P Memory[0]  // = min(A, B)
mov Q Memory[1]  // = max(min(A, C), B)
mov R Memory[2]  // = max(A, C)
```

Novelty: Copy Move

d



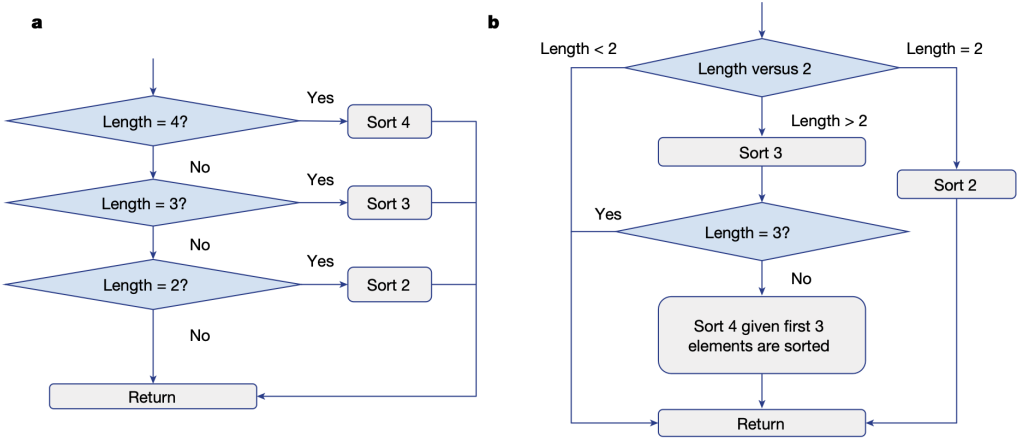
e Original

```
Memory[0] = A
Memory[1] = B
Memory[2] = C
Memory[3] = D

mov Memory[0] P  // P = A
mov Memory[1] Q  // Q = B
mov Memory[2] R  // R = C
mov Memory[3] S  // S = D

cmp S P
mov P T
cmovl S P  // P = min(A, D)
cmovl T S  // S = max(A, D)
cmp R P
mov P T
cmovg R P  // P = max(C, min(A, D))
cmovl R T  // T = min(A, C, D)
cmp Q T
mov T U
cmovl Q U  // U = min(A, B, C, D)
cmovl T Q  // Q = max(B, min(A, C, D))

mov U Memory[0]  // = min(A, B, C, D)
mov Q Memory[1]  // = max(B, min(A, C, D))
mov P Memory[2]  // = max(C, min(A, D))
mov S Memory[3]  // = max(A, D)
```

f AlphaDev

```
Memory[0] = A
Memory[1] = B
Memory[2] = C
Memory[3] = D

mov Memory[0] P  // P = A
mov Memory[1] Q  // Q = B
mov Memory[2] R  // R = C
mov Memory[3] S  // S = D

cmp S P
mov P T
cmovl S P  // P = min(A, D)
cmovl T S  // S = max(A, D)
cmp R P

cmovg R P  // P = max(C, min(A, D))
cmovl R T  // T = min(A, C)
cmp Q T
mov T U
cmovl Q U  // U = min(A, B, C)
cmovl T Q  // Q = max(B, min(A, C))

mov U Memory[0]  // = min(A, B, C, D)
mov Q Memory[1]  // = max(B, min(A, C))
mov P Memory[2]  // = max(C, min(A, D))
mov S Memory[3]  // = max(A, D)
```

Novelty: VarSort



Impact

- New insights into sorting algorithms that were previously unknown
- First updates to these sorting routines in LLVM in a decade
- 70% speedup for sorting 5 elements, 1.7% speedup for sorting $\geq$ 250,000 elements
- Power of using DRL to search a large combinatorial space of programs

Other Search Options

Extended Data Table 2 | Comparison of AlphaDev ar
AlphaDev-S for fixed sort

a

	AlphaDev-S-CS	AlphaDev-S-WS	AlphaDev
Sort3	N/S	**17**	**17**
Sort4	N/S	**28**	**28**
Sort5	N/S	**42**	**42**

b

# programs	Sort3	Sort4	Sort5
AlphaDev	446K	1.3M	6.23M
AlphaDev-S-WS	51.7M	338K	170B
AlphaDev-S-CS	77.5B	325B	1.9T

c

Wall clock time (s)	Sort3	Sort4	Sort5
AlphaDev	6 mins	30 mins	0.73 days
AlphaDev-S-WS	0.4s	0.6s	0.24 days
AlphaDev-S-CS	25 mins	1hr 6mins	0.88 days